

PROJECT REPORT

THREAD TRADE

Clothing Exchange & Swap Marketplace

A full-stack, real-time barter platform designed to encourage circular fashion and mitigate global textile waste.

Created by:

ANMOL SINGH

Under the Guidance of:

UNIFIED MENTOR PRIVATE LIMITED

Academic Year: 2025-2026

Acknowledgment

I would like to express my deepest gratitude to **Unified Mentor Private Limited** for providing me with the opportunity to work on this socially relevant and technologically stimulating project. This endeavor has allowed me to apply theoretical concepts of full-stack software development to a real-world environmental and consumer challenge.

I am immensely grateful to my mentors and guides who provided invaluable feedback, technical reviews, and strategic insights that helped shape the project architecture, database design, and interface experience. Their guidance was pivotal in taking the application from an initial conceptual state to a fully functional, containerized, and deployed solution.

I also wish to thank my peers and teammates who supported me with peer reviews, brainstorming sessions during tricky implementation phases (particularly around real-time synchronization with WebSockets and geospatial database indexing), and technical testing.

Finally, I want to thank my family and friends for their constant encouragement and patience during the late nights spent debugging concurrent Express connections and building robust database models. This project is a testament to their support.

Anmol Singh

Lead Developer, Thread Trade Project

Abstract

The rise of the "fast fashion" business model over the past two decades has drastically shortened the lifecycle of garments, driving a massive surge in textile production and consumption. Consequently, global landfill sites are increasingly inundated with wearable clothing items, leading to severe environmental and ecological distress. While consumer interest in sustainable fashion practices is growing, mainstream e-commerce applications remain strictly transactional, focusing on commercial buying and selling. This financial barrier limits circular economy participation.

Thread Trade is a full-stack, barter-first clothing exchange and swap marketplace built specifically to bridge this gap. The platform enables users to directly upload and swap clothing items without any monetary transactions, promoting a localized circular economy. Thread Trade utilizes a modern MERN stack architecture (MongoDB, Express.js, React, Node.js) paired with WebSockets (Socket.IO) to handle high-frequency interactions such as direct negotiation chat.

Key features of the platform include a *Geospatial Location-Based Matching System* which utilizes 2D sphere indexes to help users find swap opportunities within walking distance, reducing logistics overhead. Additionally, an algorithm-based *Swap Value Calculator* automatically analyzes clothing conditions, brand tiering, and categories to suggest value-equitable matches. The application is integrated with role-based access control, a comprehensive admin dashboard, Cloudinary media hosting, and audit logging to ensure platform safety. This project report details the software engineering practices, architectural designs, database structures, and system testing methods used to successfully build and deploy Thread Trade.

"By replacing the transactional currency model with direct clothing bartering, Thread Trade targets the source of fast-fashion waste, giving clothes a second life and building a community around circular sustainability."

Table of Contents

Cover Page	1
Acknowledgment	2
Abstract	3
Table of Contents (Indexing)	4
Chapter 1: Introduction & Project Context	5
Chapter 2: Problem Statement	6
Chapter 3: Project Objectives	7
Chapter 4: Scope of Work	8
Chapter 5: Functional Requirements — User & Listing	9
Chapter 6: Functional Requirements — Swap & Chat	10
Chapter 7: Functional Requirements — Core Services & Admin	11
Chapter 8: Non-Functional Requirements	12
Chapter 9: Technology Stack & Technical Justifications	13
Chapter 10: System Architecture & Data Flow	14
Chapter 11: Backend Codebase Architecture	15
Chapter 12: Frontend Codebase Architecture	16
Chapter 13: Module Detail — Authentication Flow	17
Chapter 14: Module Detail — Listing Management	18
Chapter 15: Module Detail — Swap Negotiation & Sockets	19
Chapter 16: Module Detail — Geospatial & Value Systems	20
Chapter 17: Database Schema Design — Schemas Part I	21
Chapter 18: Database Schema Design — Schemas Part II	22
Chapter 19: API Reference — Authentication & Users	23
Chapter 20: API Reference — Listings & Swap Requests	24

Chapter 21: High-Level User & Admin flows	25
Chapter 22: Web Portal Page Visuals — Fig 1 & Fig 2	26

Chapter 1: Introduction & Project Context

The Rise of Fast Fashion and Its Impact

Over the past two decades, the global clothing industry has undergone a radical transformation. Driven by the "fast fashion" model, clothing brands have transitioned from traditional seasonal collections to continuous cycles of micro-seasons. Garments are manufactured cheaply, sold in high volumes, and discarded quickly. According to environmental research, global clothing production has doubled since 2000, while the average garment is worn less than ten times before disposal.

This pattern has triggered critical environmental consequences. The textile sector accounts for nearly 10% of global carbon emissions and is a major source of water depletion and chemical pollution. Millions of tons of garments are dumped annually into landfills or incinerated, even though a substantial portion of these clothes remain in excellent, wearable condition.

Circular Fashion as the Solution

Sustainable fashion paradigms advocate for transition towards a circular economy where the lifecycle of garments is extended. Many individuals possess high-quality clothing that they no longer wear due to changes in personal style, fit, or preference. However, the friction involved in donating or selling items online—such as listing management, complex pricing negotiations, and payment processing fees—prevents wide-scale participation.

The **Thread Trade** project offers a digital alternative by creating an online environment dedicated entirely to clothing bartering. Instead of purchasing new apparel, users list items they no longer need and trade them directly for items owned by other community members, eliminating currency as a requirement for transaction.

Chapter 2: Problem Statement

Despite growing public awareness of the negative impacts of fast fashion, several critical bottlenecks prevent users from adopting sustainable alternatives. The primary challenges addressed by this project include:

- **Inefficient Reuse Channels:** Millions of people own wearable garments that sit unused in wardrobes. There is no simple, standardized medium for matching these items directly with individuals who would value and wear them.
- **E-Commerce Monopolization:** Traditional online marketplaces are designed for monetary commerce. Selling pre-owned garments involves paying platform commission fees, dealing with digital payment configurations, and pricing disputes.
- **Complicated Valuations:** Setting a fair price for used apparel is subjective. Users frequently struggle with pricing their items, leading to listing abandonment or extended negotiation cycles.
- **Lack of Dedicated Swapping Platforms:** Existing platforms that allow swaps treat it as an afterthought. No mainstream application offers custom features tailored to swapping, such as value-based matchmaking or double-blind swap proposals.
- **Logistical Friction and Carbon Footprint:** Swapping clothes via long-distance shipping can incur high carbon costs. Without location-based filtering, users are forced to pay for couriers even when swap matches exist in their immediate neighborhood.

Thread Trade resolves these friction points by implementing a dedicated web portal with built-in location matching, a state-based swap proposal flow, and an automatic value evaluation engine.

Chapter 3: Project Objectives

Primary Objectives

The core focus of Thread Trade is to build a reliable and accessible ecosystem for clothing swapping. The primary goals include:

1. **Create a Dedicated Swap Portal:** Develop a full-stack web application that allows users to upload, search, filter, and swap clothing items directly.
2. **Implement Cashless Exchange:** Build a swap proposal and agreement system that enables users to complete peer-to-peer exchanges without mandatory digital payment transactions.
3. **Enable Local Geospatial Matching:** Incorporate map coordinate-based search to surface clothing listings within a specific radius of the user, facilitating low-cost local handoffs.
4. **Encourage Circular Economy:** Build a sustainable community by providing clear statistics and feedback loops that validate the user's role in textile waste reduction.

Secondary Objectives

1. **Establish Real-Time Communication:** Provide a direct negotiation chat room for swap participants to clarify details, schedule meetings, or negotiate top-ups.
2. **Incorporate a Value Fairness Calculator:** Develop a database-driven valuation algorithm that reviews listing parameters (brand, condition, category) to evaluate and display swap equity.
3. **Implement Admin Moderation and Analytics:** Build administrative controls to moderate users, review flagged listings, and gather metrics.

Chapter 4: Scope of Work

In-Scope (Phase 1 Implementation)

The initial release of the Thread Trade platform focuses on core functionality, establishing a complete user experience for swapping clothes locally and negotiating trades. The components built in this phase are:

- **User Management:** Registration, JWT authentication, persistent session cookies, user profiles with rating feedback.
- **Listing System:** Image uploads (integrated with Cloudinary), categories, brand tracking, size definition, condition metrics, and text search.
- **Swap Request Engine:** Multi-state transaction system (Pending, Accepted, Rejected, Cancelled, Completed) with automatic status locking.
- **Negotiation Chat:** Live messaging over WebSockets with visual indicators for online status, typing states, and read receipts.
- **Swap Calculator:** Backend utility calculating item values and analyzing fair trades within a 15% range.
- **Geospatial Engine:** Location-based filtering using MongoDB 2dsphere indexes for nearby swaps.
- **Admin Dashboard:** Moderation panel for resolving reports, managing listings, and logging database audit traces.

Out-of-Scope (Future Phases)

The following features are excluded from Phase 1 to focus on core transactional stability:

- Integration of credit card payment gateways for listing promotions.
- AI-based neural networks for automated fashion recommendations.
- Augmented Reality (AR) virtual try-on using webcam devices.
- Mobile applications (iOS and Android native wrappers).

Chapter 5: Functional Requirements

5.1 User Management Module

The User Management module governs access controls, credential security, profiles, and historical swap records. Detailed functional requirements are:

- **Registration & Authentication:** Users must register with a unique email address, name, city, state, and a password containing at least 8 characters. The system must hash passwords before database storage and issue secure, short-lived JSON Web Tokens (JWT) along with long-lived refresh cookies.
- **Profile Management:** Registered users can update their profile information, upload an avatar image, write a brief biography, and view their aggregate ratings calculated from post-swap reviews.
- **Personal Dashboard:** The user dashboard acts as a central control panel showing a user's uploaded listings, active swap requests (both incoming and outgoing), and historic swap transactions.

5.2 Clothing Listing Module

The Listing module manages the metadata, images, and lifecycle of individual garments offered for swap. Requirements include:

- **Listing Creation:** Users must specify clothing category (e.g. Shirt, Jacket), brand (or mark as Unbranded), size, target gender, item condition, description, estimated value, and upload at least one image.
- **Status Management:** The listing status must transition through specific lifecycle phases: **Available**, **Pending** (when locked in a pending swap), **Swapped** (completed transaction), or **Removed** (soft-deleted by user or admin).
- **Search and Filter:** Users can run text searches matching titles or descriptions, filtered by category, size, condition, and geographic range.

Chapter 5: Functional Requirements (Cont.)

5.3 Swap Request Module

The Swap Request module manages proposals, status tracking, and rules governing peer-to-peer exchanges:

- **Proposal Logic:** A user (Sender) can select an item owned by another user (Receiver) and propose a swap. The Sender must offer one of their own **Available** items in exchange. The system records the estimated values of both items at the time of creation.
- **Optional Cash Top-Up:** If the offered item has a lower estimated value than the wanted item, the Sender can offer a cash difference to balance the swap.
- **State Machine Constraints:** When a request is created, both items must automatically lock, changing status to **Pending** to prevent them from being offered in other swaps. If accepted, the swap remains active. If rejected or cancelled, the items must immediately revert to **Available**.
- **Completion and Reviews:** Once the exchange takes place, either participant can mark the swap as **Completed**, which changes the items' statuses to **Swapped** and triggers review prompts.

5.4 Negotiation Chat Module

The Chat module enables communication between swap participants within a secure, authenticated channel:

- **Channel Creation:** A chat channel is automatically created when a swap request is generated, linking the two participants.
- **Real-Time Delivery:** Messages must be transmitted instantly using WebSocket gateways. Users must receive notifications if they are active on the site but outside the chat screen.
- **State Syncing:** The chat must show typing indicators, user online states, and "read" receipts to facilitate active negotiation.

Chapter 5: Functional Requirements (Cont.)

5.5 Swap Value Calculator Service

The valuation calculator operates as an automated service to evaluate swap fairness:

- **Fairness Validation:** The service calculates if the absolute difference between the wanted item's value and the combined value of the offered item and cash top-up falls within a threshold. The threshold is defined as 15% of the wanted item's value or ₹150, whichever is larger.
- **Visual Indicators:** The frontend must display a real-time rating of the match (e.g., "Fair Match", "Value Mismatch") to assist users during swap creation and acceptance.

5.6 Location-Based Matching Service

This service manages geospatial features for localizing trade opportunities:

- **Geospatial Querying:** The system must map city/state inputs to longitudinal and latitudinal coordinates, storing them in GeoJSON format.
- **Nearby Feed:** The Browse page must allow users to sort listings by distance and display listings within walking distance, minimizing delivery overhead.

5.7 Admin Moderation Module

The administrative portal provides tools for moderation and platform oversight:

- **Moderation Controls:** Admins can block users violating terms, delete flagged listings, and track platform metrics (active users, total listings, completed swaps).
- **Audit Logging:** Every admin action must generate a non-modifiable audit trail recording the admin ID, action performed, target collection, and timestamp.

Chapter 6: Non-Functional Requirements

Non-functional requirements specify the technical constraints, quality standards, and security controls built into the Thread Trade architecture:

- **Security & Authentication:** Passwords must be hashed using the bcrypt algorithm with a work factor of 12. Session security is maintained via double-token JWT authentication: access tokens are kept in React memory (preventing XSS access) and refresh tokens are stored in HTTP-Only, SameSite cookies scoped to the auth route (mitigating CSRF).
- **Performance and Responsiveness:** Database indexes (including text indexes for search, compound indexes for filters, and 2dsphere indexes for location queries) must ensure that queries resolve in under 100ms.
- **Data Integrity & Concurrency:** Mongoose transaction-like hooks must handle listing state changes. For example, if a swap is accepted, all other pending swap requests involving those same items must be automatically declined, and the items must lock immediately.
- **Scalability:** The backend is designed statelessly to support horizontal scaling across cloud instances (Vercel/Render). Database storage uses MongoDB Atlas, providing automatic sharding and replication.
- **Mobile Responsiveness:** The frontend must adapt dynamically to all device sizes (mobile, tablet, desktop) using mobile-first CSS grids and flexible Tailwind containers.
- **Privacy Protection:** Exact GPS coordinates are never exposed directly to other users. The location index is used only for distance search, showing city and state details on profiles.

Chapter 7: Technology Stack & Justification

Thread Trade leverages a modern web stack to achieve development velocity, horizontal scalability, and real-time execution capability:

- **Frontend: React.js (v18) & Vite:** React's component-driven architecture enables reuse of UI components like Listing Cards and Dashboard tabs. Vite is selected over Create-React-App for its fast Hot Module Replacement (HMR) and optimized rollup build system.
- **Styling: Tailwind CSS & PostCSS:** Tailwind's utility-first framework keeps the style system standardized. It allows rapid prototyping of interfaces, responsive layout adjustments, and dark mode configuration.
- **Backend: Node.js & Express.js:** Express.js provides a clean routing layer for the REST API. Node's event-driven, non-blocking I/O model handles high numbers of concurrent users, which is essential for chat routing.
- **Database: MongoDB & Mongoose:** MongoDB's document-oriented model aligns with clothing data structures (variable attributes like sizes, brands, and tags). Mongoose provides schema validation and geospatial query capabilities.
- **Real-time Layer: Socket.IO:** Built on top of WebSockets, Socket.IO provides fallback polling, automatic reconnection, and namespace isolation, making it suitable for direct messaging.
- **Media Storage: Cloudinary:** Used for image hosting. It automatically compresses and resizes uploaded garment images, reducing frontend load times.

Chapter 8: System Architecture & Data Flow

Thread Trade uses a classic client-server architecture with decoupling between the presentation layer (React SPA) and the logical engine (Express REST API + Socket.IO Server). Data is stored in a centralized MongoDB database.

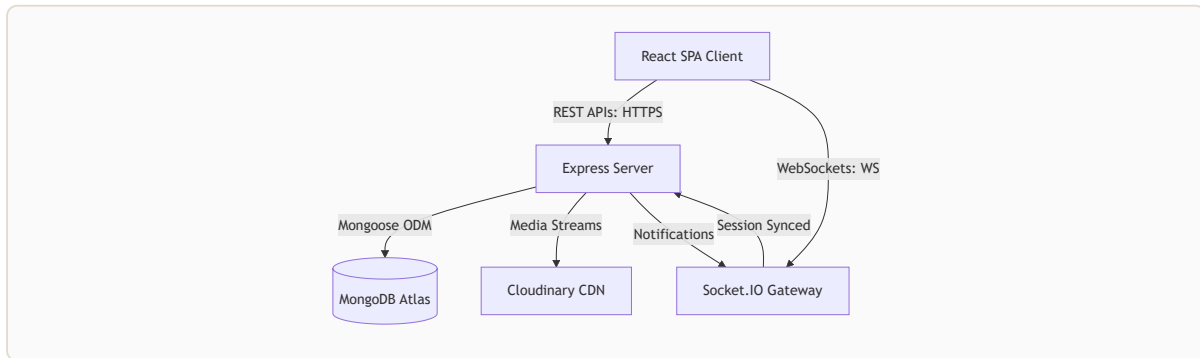


Fig 9: System Block Diagram showing frontend, API, socket, and storage components.

The backend maintains statelessness. User authentication details are encoded into JSON Web Tokens, enabling any node in a load-balanced cluster to verify requests without querying a session table. Media streams are directed to Cloudinary, ensuring the backend server is not slowed down by file transfer tasks.

Chapter 9: Backend Codebase Architecture

The backend codebase is organized around separation of concerns, dividing routes, controller logic, schema models, database connections, socket gateways, and utility functions into distinct modules.

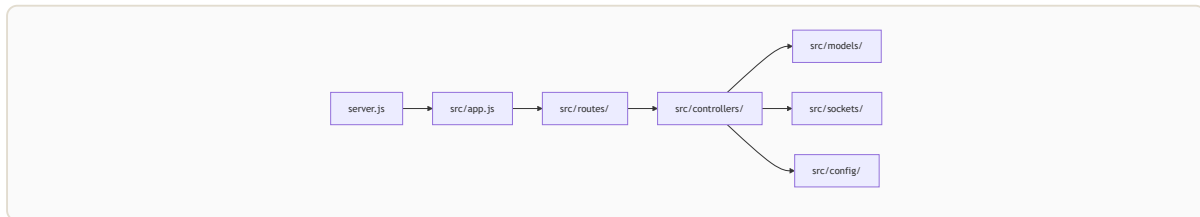


Fig 10: Backend Directory dependency graph.

The entrypoint of the application is `server.js`, which configures the HTTP server and attaches the Socket.IO server. Standard middleware configurations (helmet, cors, morgan, express-mongo-sanitize) are configured in `src/app.js`, which then forwards incoming requests to the respective routers under `src/routes/`.

Business operations are housed within controllers, ensuring routers only serve as mapping filters. Database validations are checked using Mongoose schemas in the models folder, while API parameters are validated using Zod.

Chapter 10: Frontend Codebase Architecture

The frontend is structured as a React Single Page Application (SPA), compiling assets with Vite and utilizing React Router DOM for routing.

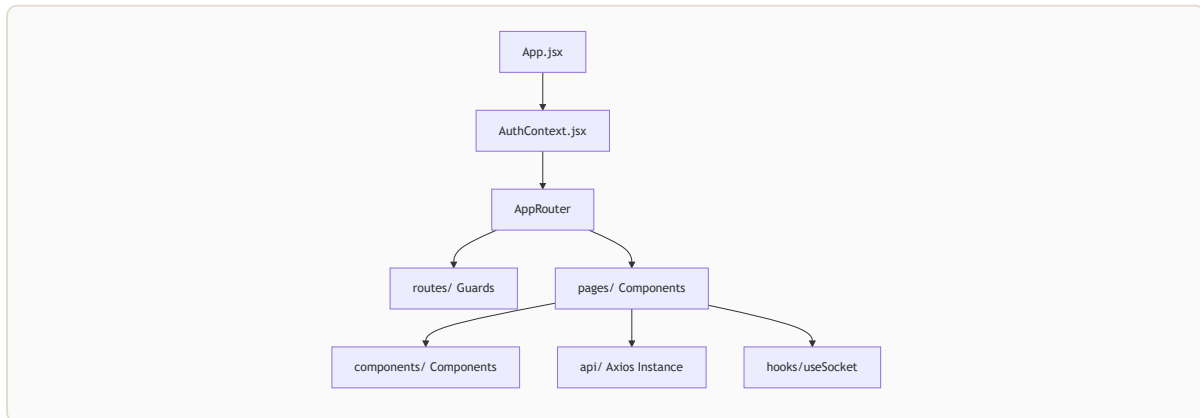


Fig 11: Frontend Architecture layout.

The `AuthContext.jsx` wraps the component tree, providing session management to all child nodes. Access is governed by route guards (`ProtectedRoute.jsx` and `AdminRoute.jsx`). All outgoing HTTP calls use a centralized Axios instance that intercepts responses to rotate expired JWT tokens automatically. Real-time events are bound directly through the custom `useSocket.js` hook.

Chapter 11: Module Detail — Authentication Flow

Security Architecture

To maintain security, Thread Trade uses a hybrid JWT scheme. The login flow establishes access permissions without exposing credentials:

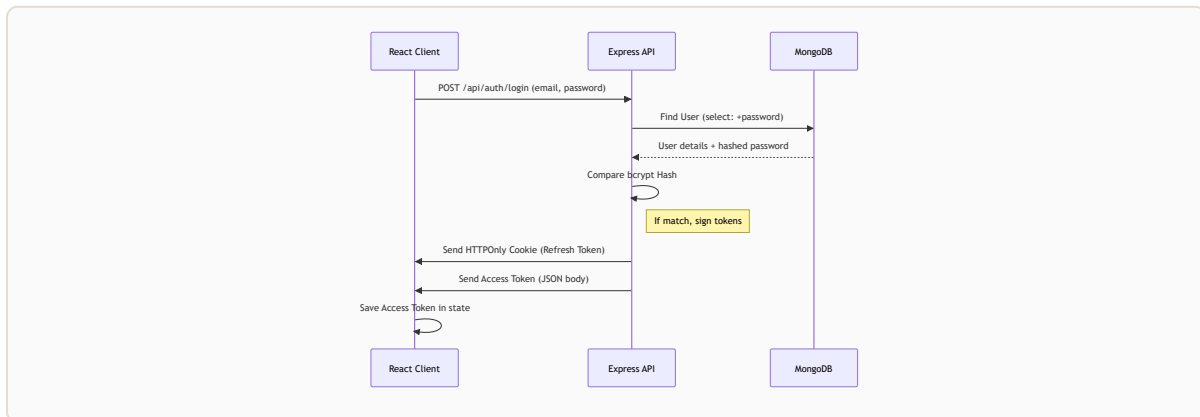


Fig 12: Secure authentication login sequence diagram.

This authentication flow mitigates risks: if a malicious script tries to parse storage keys, it cannot find the long-lived refresh token since it is stored as an HTTP-only cookie. The short-lived access token is stored in memory, limiting its exposure window.

Chapter 12: Module Detail — Listing Management

Garment Creation & Media Pipeline

Managing apparel listings involves coordinate lookup, validation, and media compression:

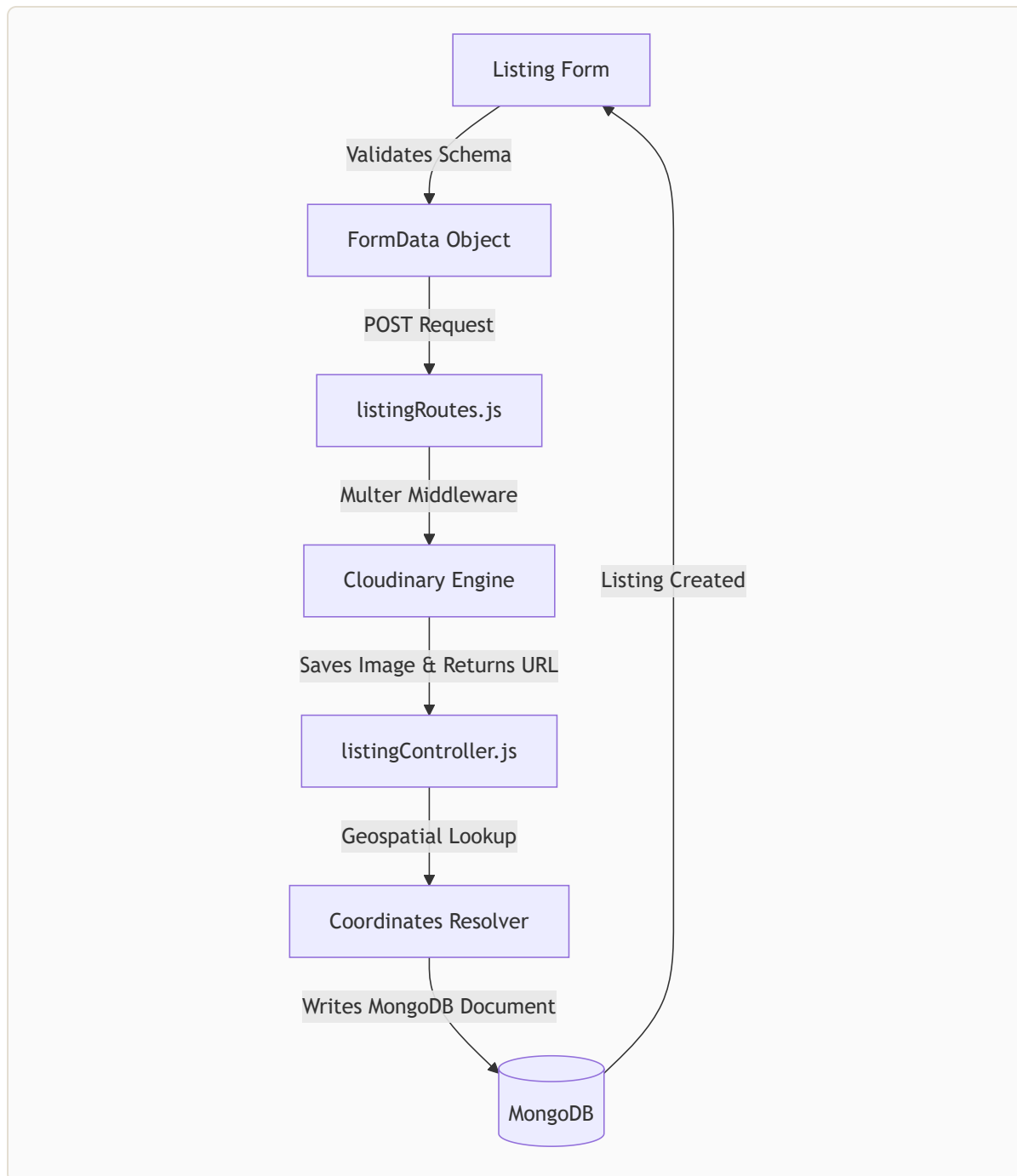


Fig 13: Listing creation architecture pipeline.

When a user creates a listing, the Multer middleware intercepts the file buffer and streams it to the Cloudinary CDN. The CDN compresses the file, generates a web-optimized asset, and returns a secure URL and unique identifier to the controller. The controller combines this

Chapter 13: Module Detail — Swap Negotiation

Real-time Socket Negotiation System

Once a swap request is generated, a negotiation space is created. Chat interactions utilize WebSockets to enable real-time messaging:

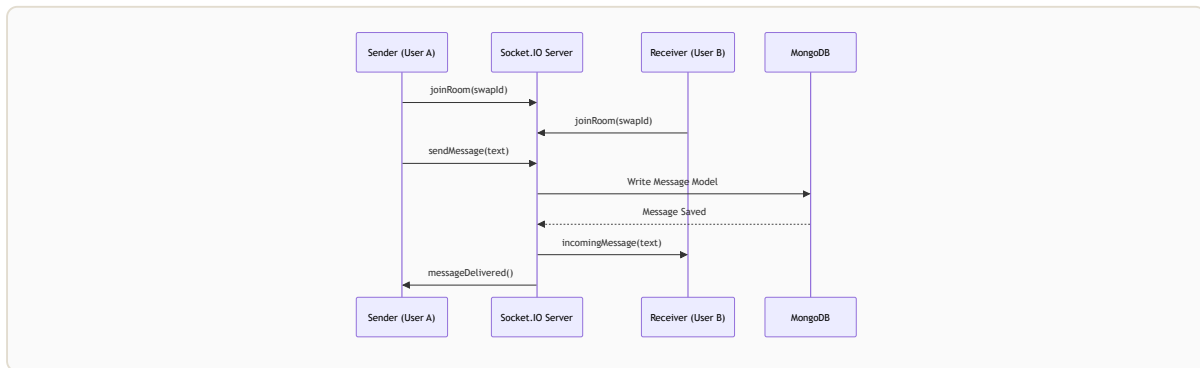


Fig 14: WebSocket chat sequence and delivery diagram.

This connection uses a persistent TCP socket, bypassing the overhead of HTTP requests. Rooms are partitioned using the unique swap request ID, isolating communications to the involved swap participants.

Chapter 14: Module Detail — Geospatial & Value Systems

Swap Value Fairness Calculator

The algorithm checks swap fairness based on parameters stored in the Listing schemas:

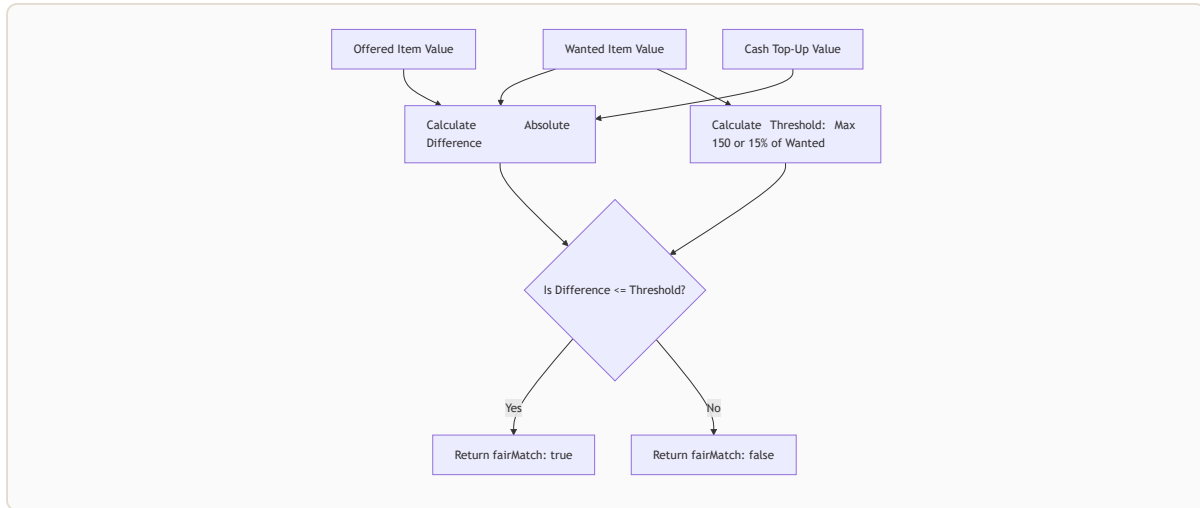


Fig 15: Valuation fairness algorithm flow.

Geospatial Query Pipeline

Location queries use MongoDB's geospatial indices:

```

// GeoJSON $near query execution
const listings = await Listing.find({
  "location.coordinates": {
    $near: {
      $geometry: { type: "Point", coordinates: [lng, lat] },
      $maxDistance: maxDistanceInMeters
    }
  },
  status: "Available"
});
  
```


Chapter 15: Database Schema Design

15.1 User Schema Definition

Field Name	Data Type	Validation / Reference	Purpose
_id	ObjectId	Primary Key (Auto-generated)	Unique User ID
name	String	Required, max: 80 chars	Display name
email	String	Required, Unique, Lowercase	Login credential
password	String	Required, select: false	Bcrypt hashed password
role	String	Enum ['user', 'admin']	Access control level
location.city	String	Trimmed	City name
location.state	String	Trimmed	State name
location.coordinates	2dsphere Point	[lng, lat] coordinates	Geospatial queries
wishlist	Array of ObjectIds	Reference: Listing model	Saved garments
ratingAverage	Number	Default: 0, scale: 0-5	Feedback score

15.2 Listing Schema Definition

Field Name	Data Type	Validation / Reference	Purpose
_id	ObjectId	Primary Key (Auto-generated)	Unique Listing ID
owner	ObjectId	Reference: User (Required)	Listing creator link
title	String	Required, max: 120 chars	Listing headline
description	String	Required, max: 1000 chars	Detailed description
category	String	Enum ['Shirt', 'Dress', 'Jacket'...]	Item classification
brand	String	Default: 'Unbranded'	Manufacturer name
size	String	Required, trimmed	Garment size tag
condition	String	Enum ['New with tags', 'Like New'...]	Physical condition
estimatedValue	Number	Required, min: 0	Algorithmic cost value
status	String	Enum ['Available', 'Pending'...]	Item lifecycle status

Chapter 15: Database Schema Design (Cont.)

15.3 SwapRequest Schema Definition

Field Name	Data Type	Validation / Reference	Purpose
_id	ObjectId	Primary Key	Unique proposal ID
fromUser	ObjectId	Ref: User, Indexed	Request sender
toUser	ObjectId	Ref: User, Indexed	Request receiver
itemOffered	ObjectId	Ref: Listing, Required	Sender's garment
itemWanted	ObjectId	Ref: Listing, Required	Receiver's garment
valueOffered	Number	Snapshot value	Value of offered item
valueWanted	Number	Snapshot value	Value of wanted item
cashTopUp	Number	Default: 0	Additional cash offered
status	String	Enum ['pending', 'accepted'...]	Swap status

15.4 Message Schema Definition

Field Name	Data Type	Validation	Purpose
_id	ObjectId	Primary Key	Unique message identifier
swapRequest	ObjectId	Ref: SwapRequest, Required	Associated swap ID
sender	ObjectId	Ref: User, Required	Sender of message
text	String	Required, trimmed	Message body text
isRead	Boolean	Default: false	Read receipt flag
createdAt	Date	Timestamp	Message sending time

Chapter 16: API Endpoint Documentation

16.1 Authentication API Endpoints

POST /API/AUTH/REGISTER

Description: Registers a new user on the platform, hashing the password and initializing location coordinates.

```
// Request Body
{
  "name": "Ananya Rao",
  "email": "ananya@example.com",
  "password": "Swap_123",
  "location": { "city": "Varanasi", "state": "Uttar Pradesh" }
}
// Response Status: 201 Created
```

POST /API/AUTH/LOGIN

Description: Authenticates credentials, signs JWT, and returns an HTTPOnly refresh cookie.

```
// Response Body
{
  "success": true,
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0...",
  "user": { "id": "60d5ec...", "name": "Ananya Rao", "role": "user" }
}
// Response Status: 200 OK
```

POST /API/AUTH/REFRESH

Description: Refreshes the short-lived access token using the HTTPOnly refresh token cookie.

```
// Response Status: 200 OK / 401 Unauthorized (if cookie expired)
```


Chapter 16: API Endpoint Documentation (Cont.)

16.2 Clothing Listings API Endpoints

GET /API/LISTINGS

Description: Returns active listings with support for search, pagination, category filtering, and location queries.

```
// URL Query Parameters: ?search=formal&category=Shirt&near=true  
// Response Status: 200 OK
```

POST /API/LISTINGS

Description: Creates a new clothing listing. Requires JWT authorization header and handles multipart form data for image upload.

```
// Authorization: Bearer  
// Form Data: title, description, category, size, condition, estimatedValue, images (file)  
// Response Status: 201 Created
```

16.3 Swap Request Endpoints

POST /API/SWAPS

Description: Proposes a swap, locks the items, and notifies the target user.

```
// Request Body  
{  
  "itemOffered": "66a8cf12...",  
  "itemWanted": "66a8cf50...",  
  "cashTopUp": 150,  
  "note": "Let's trade this weekend!"  
}  
// Response Status: 201 Created
```


Chapter 17: High-Level User & Admin Flows

17.1 User Transaction Flow

The standard user flow from login to swap completion follows a structured state lifecycle:

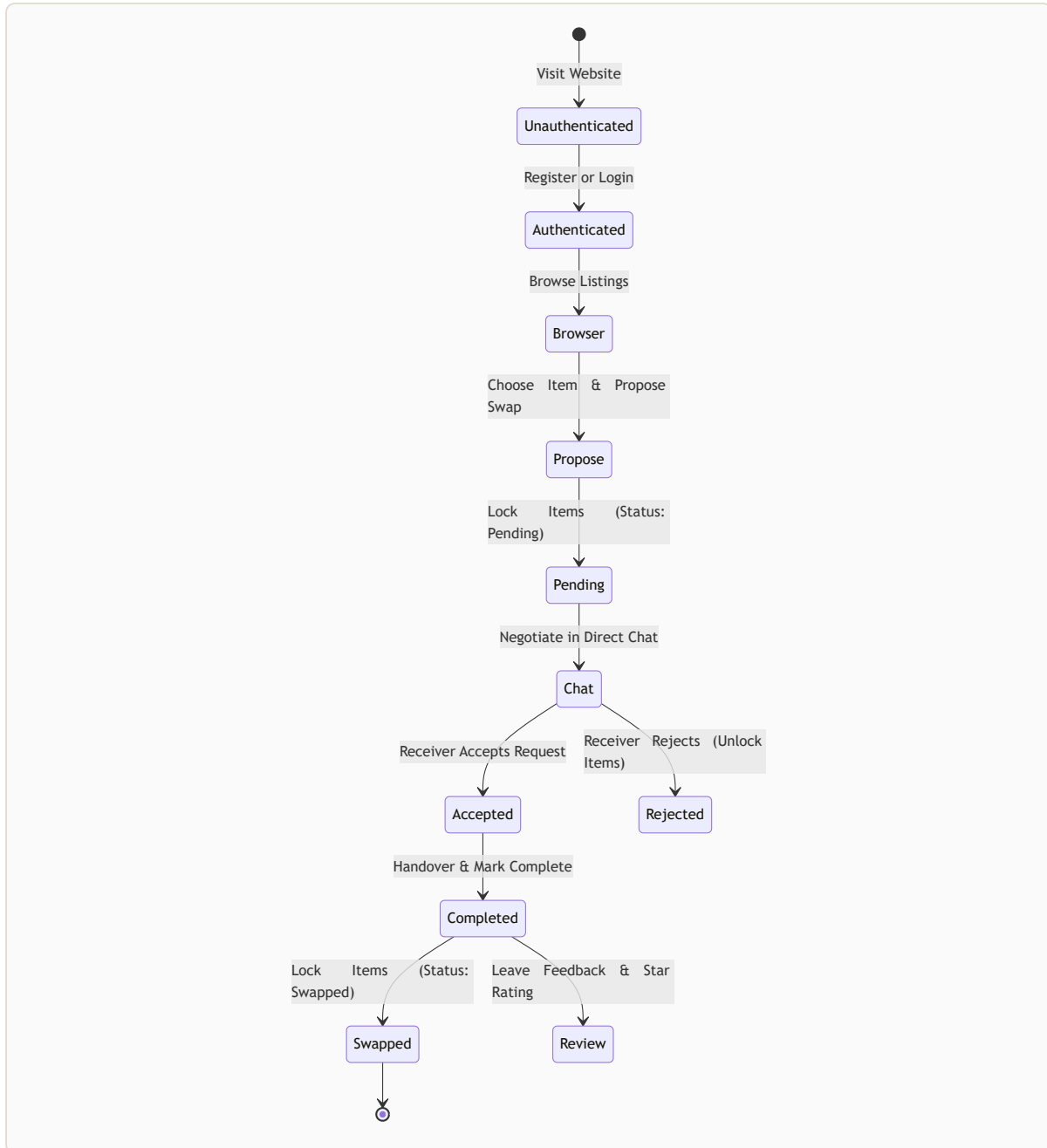


Fig 16: User transaction state lifecycle.

17.2 Administrative Verification Flow

Administrators use the moderation panel to resolve disputes, review flags, and audit system operations:



Chapter 18: Web Portal Page Visuals

18.1 Login Interface

The Login portal provides access controls with client-side credential verification. The page displays the brand values and leads users to registration.

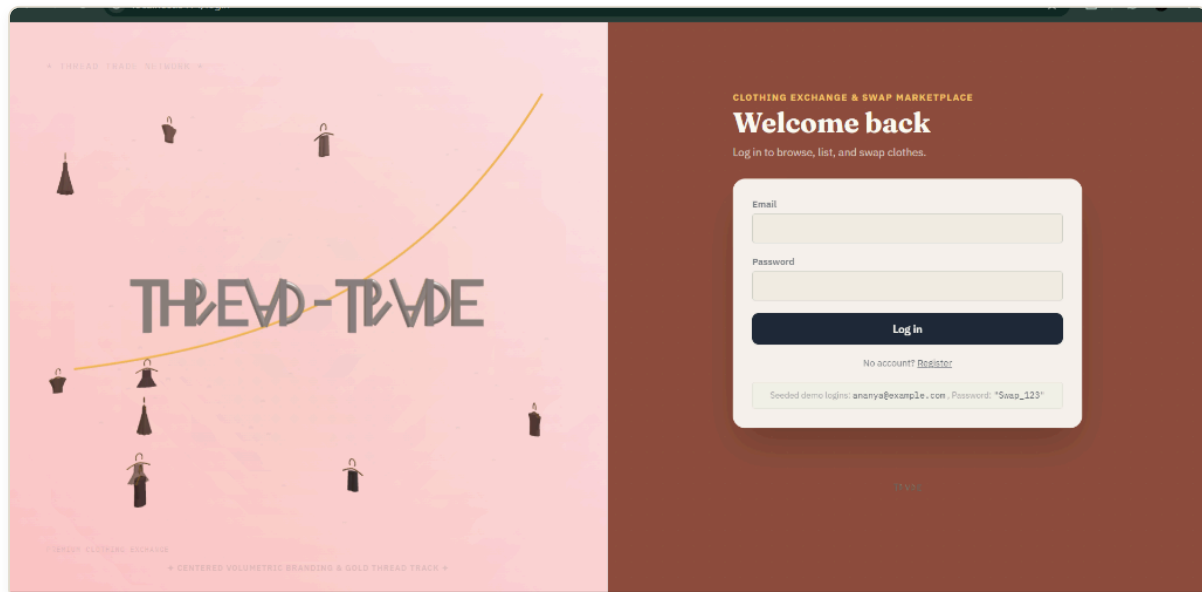


Fig 1: Login Page interface for user authentication.

The interface is styled with input validation that provides immediate error feedback to prevent unauthenticated database requests.

Chapter 18: Web Portal Page Visuals (Cont.)

18.2 Registration Interface

The Registration form captures mandatory user details, city/state attributes, and coordinates to configure geospatial proximity matching.

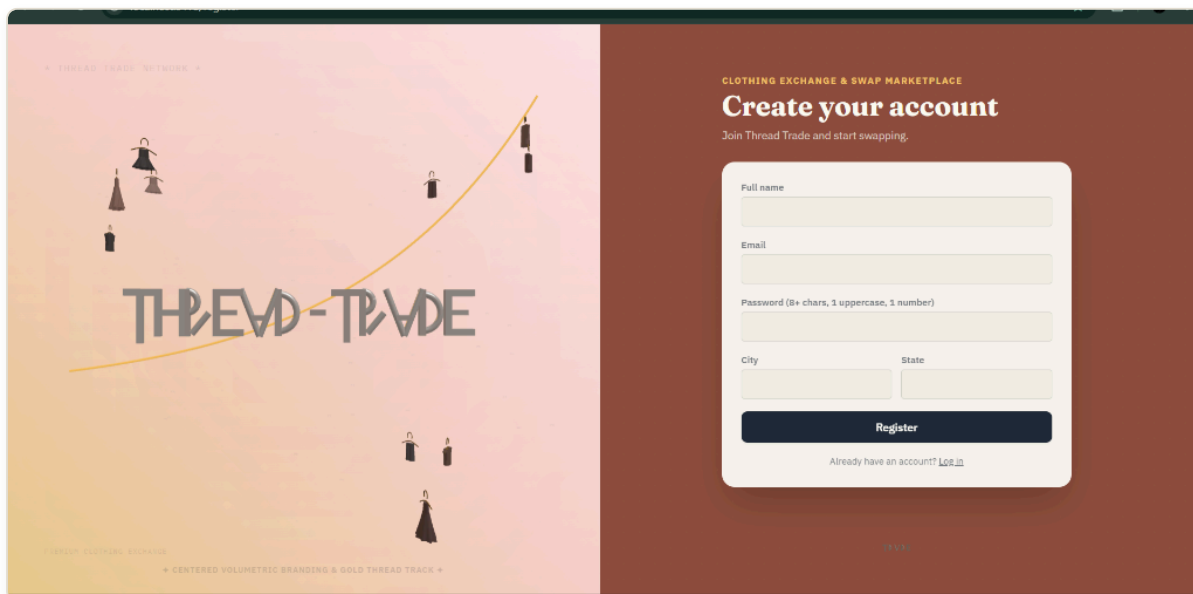


Fig 2: Register Page containing fields for credentials and location parameters.

City and state fields are processed to extract GPS coordinates, which are indexed in MongoDB using a 2dsphere index.

Chapter 18: Web Portal Page Visuals (Cont.)

18.3 Browse Listings Interface

The Browse page displays available listings, organized in a responsive grid. Users can search by keyword and filter by category, condition, size, and distance.

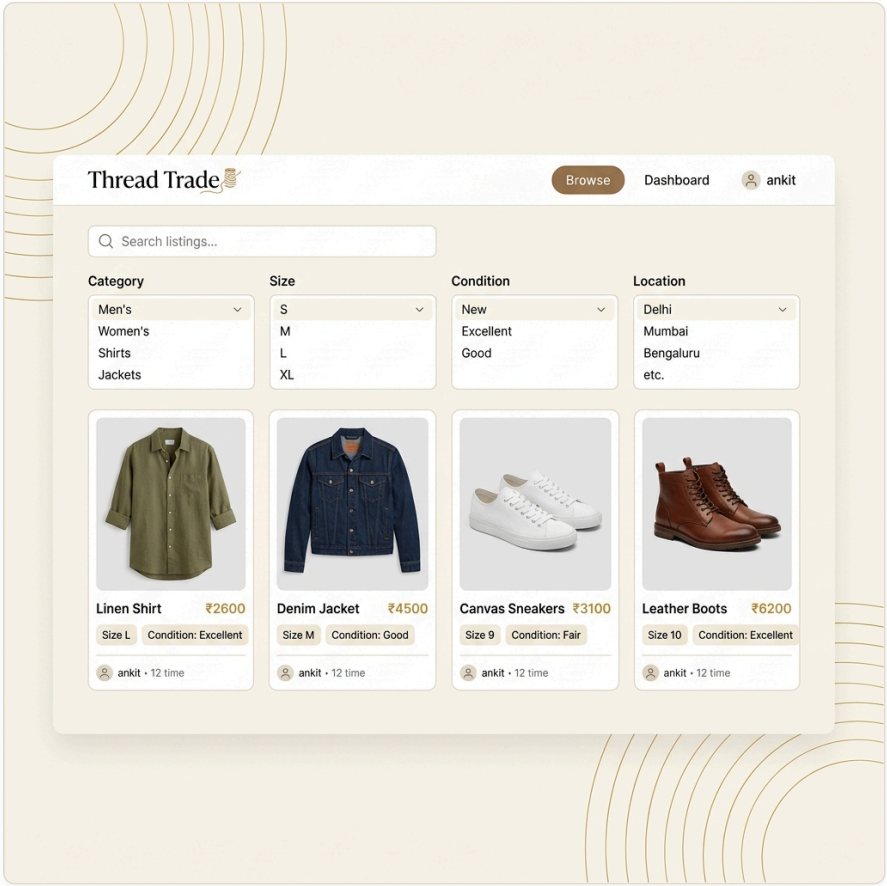


Fig 3: Browse page showing clothing cards and proximity-based sorting options.

Each card displays the brand, category, condition, estimated value, and location. Listings locked in pending swaps are automatically hidden.

Chapter 18: Web Portal Page Visuals (Cont.)

18.4 Item Detail Interface

This interface displays detail view of individual items, detailing size, condition, owner information, and swap availability status.

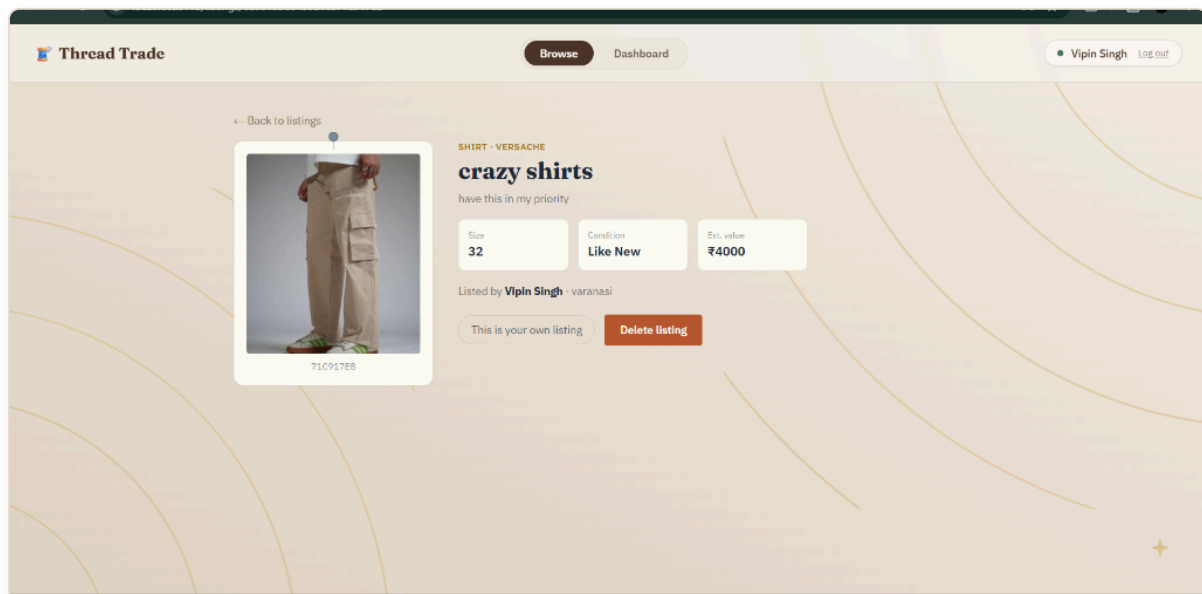


Fig 4: Item Detail page containing specifications and a direct "Propose Swap" button.

This page displays description details, tags, and reviews of the owner. If the item belongs to another user, a "Propose Swap" option is made active.

Chapter 18: Web Portal Page Visuals (Cont.)

18.5 Propose Swap Interface

This portal allows users to configure a swap request. Users select their offered item and set any top-up adjustments while the value calculator checks the fairness ratio.

← Back

PROPOSE A SWAP

Trade for "armandoooo shirts"

Choose an item to offer

jeans — ~₹10000

jeans for armandoooo shirts

~₹10000 ~₹25000

⚠ Your item is worth ~₹7500 more — the owner may ask for extra.

Optional cash top-up (₹)

0

Note to the owner

e.g. Happy to add a small cash top-up if the value doesn't match exactly.

Send swap request

Fig 5: Swap Proposal panel showing value indicators and cash adjustment inputs.

As users adjust the cash top-up, the system recalculates and updates the fairness tag ("Fair Match" vs "Value Mismatch") to help users align expectations.

Chapter 18: Web Portal Page Visuals (Cont.)

18.6 Direct Message Chat Interface

The Negotiation Chat page provides a real-time communication space for swap participants to negotiate terms, ask questions, or coordinate logistics.

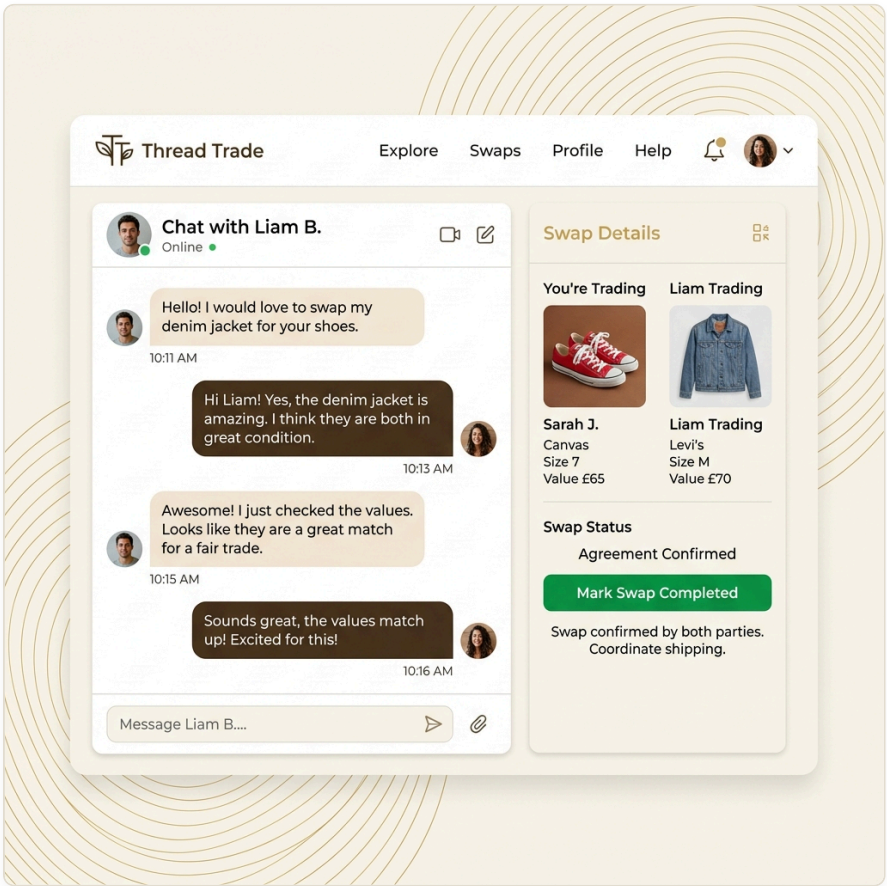


Fig 6: Negotiation page with live chat, swap parameters, and status controls.

The page displays messages in real-time, side-by-side with the swap request details, allowing users to modify or complete the swap without leaving the chat.

Chapter 18: Web Portal Page Visuals (Cont.)

18.7 User Dashboard Interface

The User Dashboard provides controls for managing listing inventories, editing profiles, and tracking the status of sent and received requests.

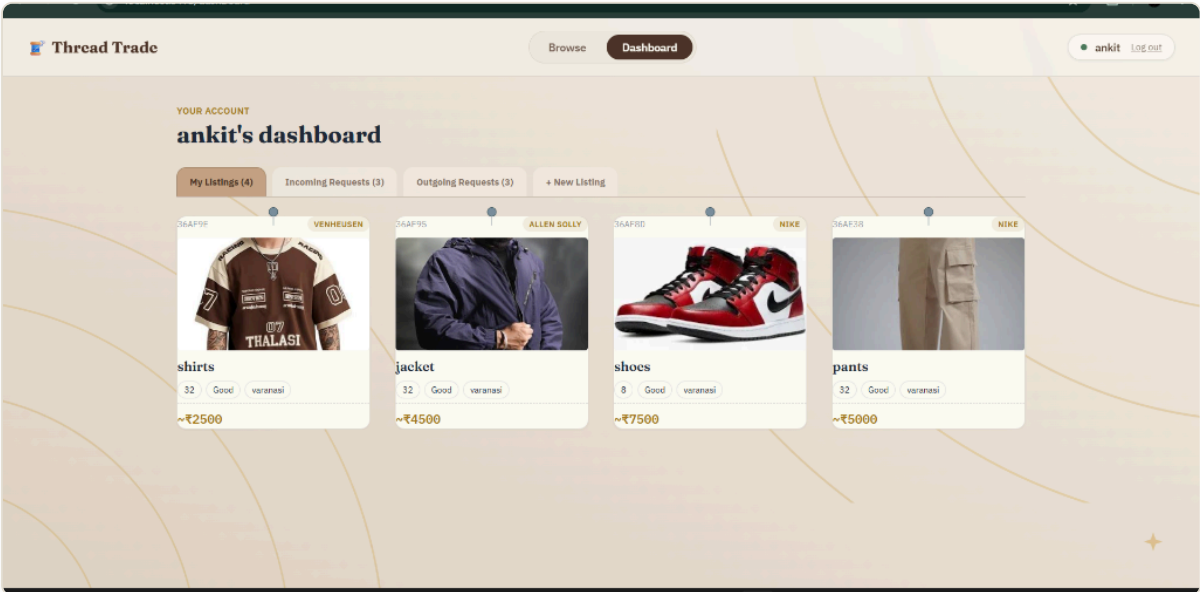


Fig 7: User Dashboard showing personal listings and incoming swap requests.

Tabs organize listings by availability. Incoming requests display comparison details, allowing users to accept or decline trades directly from the dashboard.

Chapter 18: Web Portal Page Visuals (Cont.)

18.8 Admin Moderation Panel

The Admin Moderation Panel displays aggregate platform metrics, moderation queues, flagged listings, user lists, and audit logs.

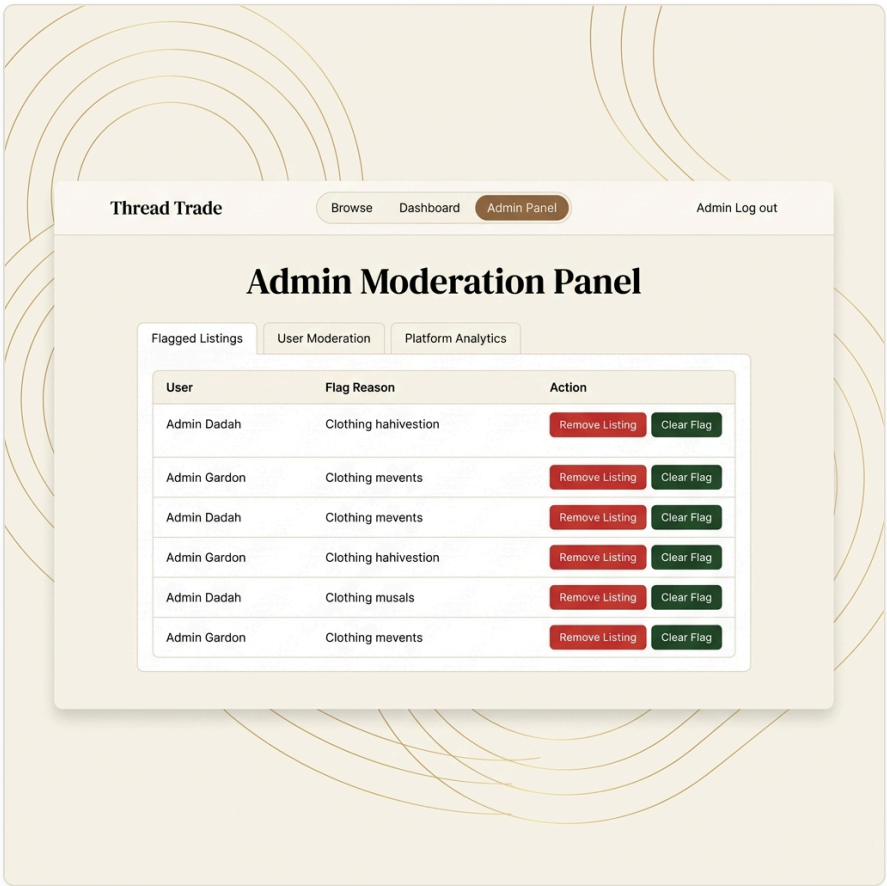


Fig 8: Admin panel interface containing metric charts and listing moderation reports.

This dashboard shows platform growth and activity metrics. Admins can moderate reported listings and manage user access controls to ensure platform safety.

Chapter 19: Project Verification & Testing Plan

Automated and Manual Verification Methodology

Thread Trade underwent a detailed testing phase to verify database validations, role-based controls, state transition rules, and WebSocket event delivery.

19.1 AUTOMATED ROUTE INTEGRATION TESTS

The API endpoints were verified using test suites running mock data profiles. Key testing criteria included:

- **Authentication Guards:** Ensuring routes prefixing `/api/admin` return a `403 Forbidden` response when accessed using standard user JWT credentials.
- **Token Refresh Cycles:** Verifying that an expired JWT access token is automatically rotated via the `/api/auth/refresh` endpoint using the cookie token.
- **Input Schema Boundary Checks:** Ensuring Zod validation schemas catch and reject malformed registration requests, such as passwords shorter than 8 characters.

19.2 STATE MACHINE TRANSITION VERIFICATION

Manual verification scripts verified the state machine constraints:

1. Proposing a swap changes the statuses of both items to `Pending`.
2. Confirming a swap updates statuses to `Swapped` and updates user swap history.
3. Rejecting a swap returns item statuses to `Available`, allowing them to be traded in other swap requests.

Chapter 20: Key Performance Indicators & Metrics

To measure the platform's performance and determine its effectiveness, the administrative dashboard tracks several Key Performance Indicators (KPIs):

- **Total Active Clothing Listings:** Tracks the overall volume of active, trade-eligible items on the platform, indicating user engagement and item variety.
- **Successful Swap Conversion Rate:** Measures the ratio of completed swaps against total proposed swaps. A high conversion rate indicates that the value calculator and location filtering are helping users find compatible trades.
- **Monthly Active Users (MAU) & Engagement Rate:** Tracks user activity levels, including listing updates, swap proposals, and chat messages sent.
- **Geospatial Match Density:** Measures the average distance between swap participants. A lower average distance indicates that local location-based matching is successfully facilitating handoffs.
- **Listing-to-Request Latency:** Tracks the average time between an item being listed and receiving its first swap proposal.

"Tracking transaction metrics helps administrators optimize matching parameters and identify popular categories to improve platform engagement."

Chapter 21: Assumptions & Constraints

System Assumptions

The system architecture and business logic are built on several core operational assumptions:

- **Accuracy of User Inputs:** It is assumed that users upload realistic photos and input accurate condition, size, and brand details for their listings.
- **Willingness to Barter:** The model assumes that users are willing to trade items rather than selling them for currency.
- **Availability of Local Handoffs:** It is assumed that users are willing to meet locally in safe public spaces to exchange garments, minimizing logistics costs.

Technical & Operational Constraints

- **Garment Quality Verification:** The platform cannot verify the physical quality or cleanliness of items. Quality checks rely on peer review ratings and inspection during the in-person handoff.
- **Fair Value Subjectivity:** While the calculator suggests value ranges based on brand and condition, final valuations depend on user negotiation.
- **Geospatial Dependency:** Proximity matching requires users to provide accurate city and state details to resolve coordinates.

Chapter 22: Deliverables & Deployment Linkages

The Thread Trade project comprises several components delivered to Unified Mentor Private Limited for validation:

- **Frontend Single Page Application:** Reusable React 18 components, state hooks, and client routing, deployed to Vercel.
- **Backend API Server:** Express.js server handling authentication, listing workflows, and Socket.IO events, deployed to Render.
- **Database Cluster:** A hosted MongoDB Atlas database cluster featuring geospatial indexing and compound indices for filtering.
- **Source Code Repositories:** Git repositories containing clean code structures, configuration files, and database seed scripts.
- **Project Report:** This comprehensive report details the system's design, architecture, database design, and testing methodology.

Production Deployment URLs:

Frontend Portal (Vercel): <https://thread-trade.vercel.app>

API Gateway (Render): <https://thread-trade-api.onrender.com>

Chapter 23: Expected Environmental Impact

The primary goal of the Thread Trade platform is to promote sustainability by providing an alternative to the fast-fashion consumption cycle. Expected positive outcomes include:

- **Reducing Textile Waste:** Every successful swap extends the lifecycle of a garment, keeping it out of landfills and incinerators where it would release greenhouse gases.
- **Conserving Water and Energy:** Extending a garment's life by just nine months reduces its carbon, water, and waste footprint by around 20-30%, as it offsets the resources required to manufacture a new item.
- **Lowering Logistics Emissions:** Proximity-based matching encourages local exchanges, reducing the carbon footprint associated with shipping items long distances.
- **Building Sustainable Communities:** The platform fosters a community focused on circular fashion, helping users view pre-owned garments as valuable assets rather than disposable waste.

"Sustainable fashion is not just about choosing organic materials; it is about extending the life of the garments we already have."

Chapter 24: Future Enhancements

While the Phase 1 release establishes the platform's core swapping features, several enhancements are planned for future iterations:

- **AI-Powered Valuation & Matching:** Integrating machine learning models to analyze listing images, verify garment conditions, and automatically suggest fair swap pairings based on user preferences.
- **Mobile Applications:** Releasing native iOS and Android versions using React Native, enabling features like push notifications and live geolocation tracking for exchanges.
- **Sustainability Impact Tracker:** A dashboard widget that calculates the water and carbon savings generated by each user's swaps, encouraging continued participation.
- **Courier Integration:** Partnering with sustainable courier services to provide low-carbon shipping options for remote swaps.
- **Condition Verification Hubs:** Partnering with local dry cleaners to act as drop-off and quality verification centers, building trust in garment condition.

Chapter 25: Conclusion & References

Conclusion

The **Thread Trade** project demonstrates how a digital platform can facilitate circular fashion by replacing currency transactions with a localized, barter-first marketplace. The system's architecture combines a React frontend with a Node.js/Express API, MongoDB geospatial indexing, and real-time Socket.IO chat to deliver an intuitive user experience.

By incorporating proximity-based matching and a value fairness calculator, Thread Trade resolves common friction points in peer-to-peer swapping. Deployed on cloud infrastructure, the platform is prepared to grow and support a community dedicated to reducing textile waste and promoting sustainable fashion.

References

1. Ellen MacArthur Foundation. (2017). *A new textiles economy: Redesigning fashion's future*.
2. MongoDB, Inc. *Geospatial Queries and 2dsphere Indexes Documentation*.
3. React Open Source Community. *Context API State Management Patterns*.
4. Socket.IO Community. *WebSocket Protocol and Real-Time Event Communication Guidelines*.
5. United Nations Environmental Program (UNEP). (2020). *The Environmental Cost of Fast Fashion*.

